



Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

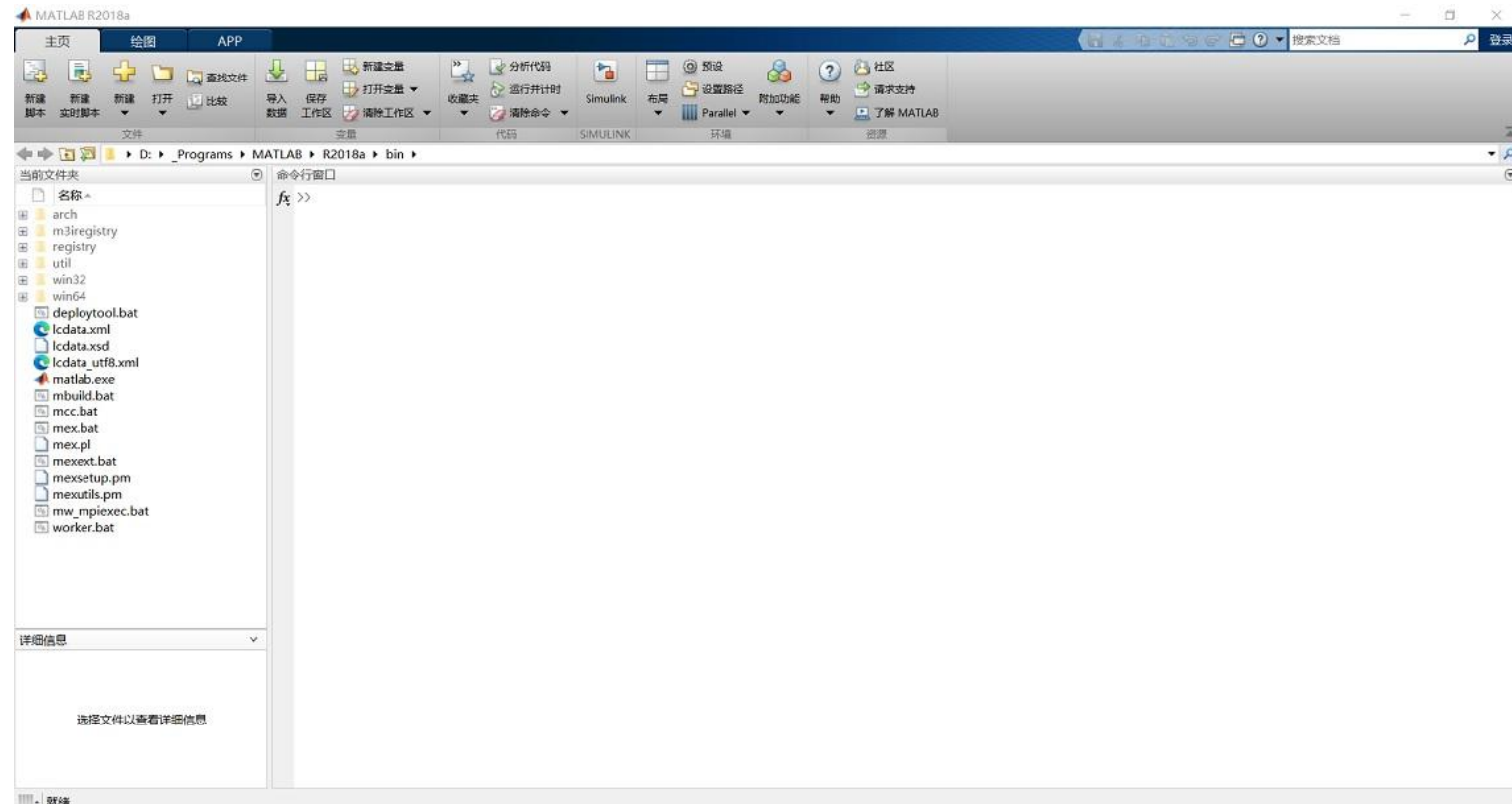
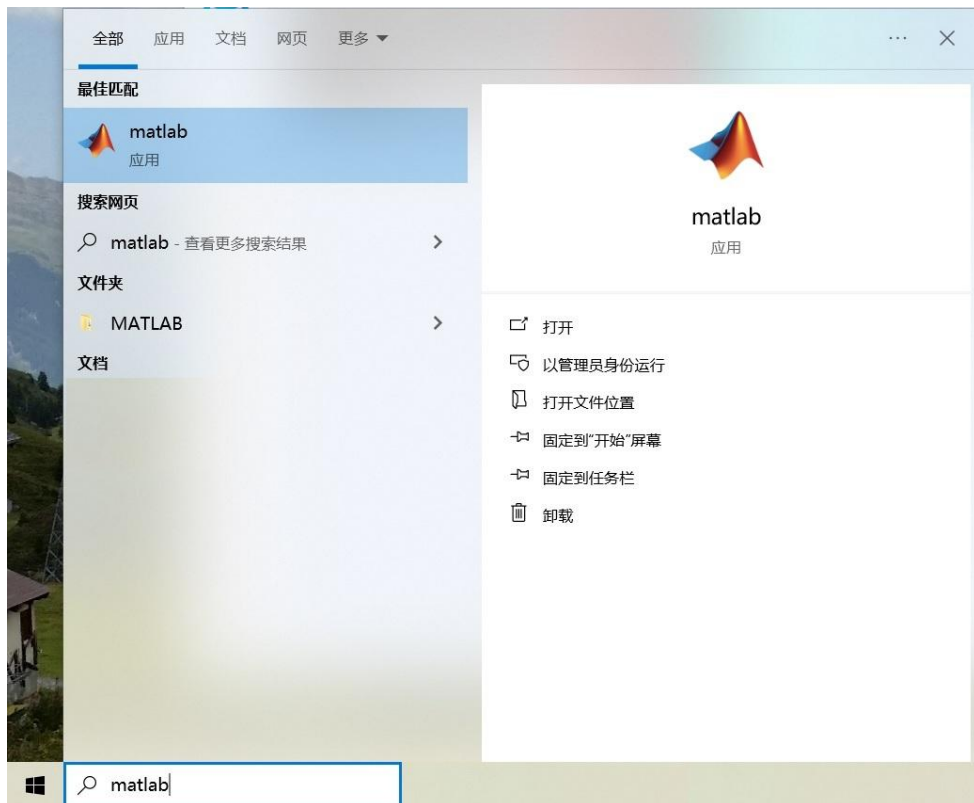
ISE 333: User Interface Development (Design) Practice Overview

Spring 2026

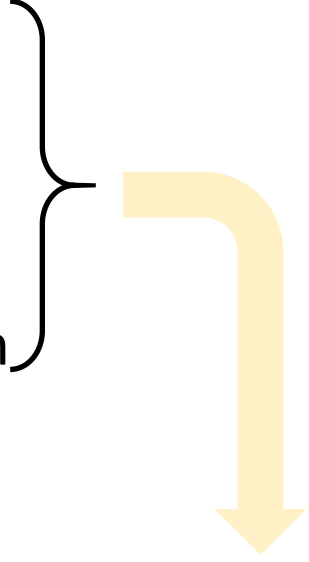
Instructor: Hao Li

Practice Overview

- Why Matlab?
 - A powerful, comprehensive, and suitable platform for teaching purpose



Practice Overview

- Why Matlab?
 - A powerful, comprehensive, and suitable platform for teaching purpose
 - Practice Topics
 - Matlab basics
 - UI-oriented image processing
 - UI-oriented computational geometry
 - Projective geometry and camera model
 - UI-oriented simple simulation/animation generation
 - Linux/CLI UI (contingent on future conditions)
 - Practice Objective
 - Obtain a basic working knowledge of graphical UI primary development as well as secondary development
- 



Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

ISE 333: User Interface Development (Design)

Practice 1: Matlab Basics

Spring 2026

Instructor: Hao Li

Numbers

- Directly-Specified Constant Numbers

```
命令行窗口
>> 20260407
ans =
    20260407
>> 333
ans =
    333
>> 5.2
ans =
    5.2000
>> -1.414
ans =
   -1.4140
>> 3.1415926
ans =
    3.1416
>> pi
ans =
    3.1416
```

```
>> format long
>> 3.1415926
ans =
    3.1415926000000000
```

Special symbols

```
>> format long
>> pi
ans =
    3.141592653589793
>> format short
>> pi
ans =
    3.1416
```

Numbers

- Arithmetic Operations

- Addition +
- Subtraction -
- Multiplication *
- Division / or \
- Power ^ or power()
- Root sqrt, ^ or power()
- Scientific notation

```
>> 3+4
ans =
     7
>> 3-4
ans =
    -1
>> 3*4
ans =
    12
>> 3/4
ans =
    0.7500
>> 4\3
ans =
    0.7500
```

```
>> 3^4
ans =
    81
>> power(3, 4)
ans =
    81
>> sqrt(81)
ans =
     9
>> 81^(1/2)
ans =
     9
>> 81^(1/4)
ans =
     3
>> power(81, 1/4)
ans =
     3
>> 1.2345*10^6
ans =
    1234500
>> 1.2345*10^(-6)
ans =
    1.2345e-06
```

Numbers

- Vectors

- Row vectors [num1, ... , num_k]
 - Separated by ,

- Column vectors [num1; ... ; num_k]
 - Separated by ;

- Regular vectors

- zeros()
- ones()
- Arithmetic progression

```
>> [3, 1, 4, 1, 5]
ans =
     3     1     4     1     5
>> [3; 1; 4; 1; 5]
ans =
     3
     1
     4
     1
     5
```

```
>> zeros(1,3)
ans =
     0     0     0
>> zeros(3,1)
ans =
     0
     0
     0
>> ones(1,3)
ans =
     1     1     1
>> ones(3,1)
ans =
     1
     1
     1
```

```
>> 3:8
ans =
     3     4     5     6     7     8
>> 8:-1:3
ans =
     8     7     6     5     4     3
>> 2:3:11
ans =
     2     5     8    11
>> 11:-3:2
ans =
    11     8     5     2
>> transpose(3:8)
ans =
     3
     4
     5
     6
     7
     8
>> (3:8)'
ans =
     3
     4
     5
     6
     7
     8
```

Numbers

- Matrices

- [A11, ... , A1n; ; Am1, ... , Amn]
 - Elements separated by ,
 - Rows separated by ;

- Coding line continuing mark ...

- Transpose

- Attention: For complex matrices, transpose() and ' mean different things

```
>> [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
```

```
ans =
```

```
1     2     3     4
5     6     7     8
9     10    11    12
```

```
>> [1, 2, 3, 4; ...
```

```
5, 6, 7, 8; ...
```

```
9, 10, 11, 12]
```

```
ans =
```

```
1     2     3     4
5     6     7     8
9     10    11    12
```

← ... means continuing the line

```
>> transpose([1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12])
```

```
ans =
```

```
1     5     9
2     6    10
3     7    11
4     8    12
```

```
>> [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]'
```

```
ans =
```

```
1     5     9
2     6    10
3     7    11
4     8    12
```

```
>> [1+i, 2+i; 3+i, 4+i]
```

```
ans =
```

```
1.0000 + 1.0000i  2.0000 + 1.0000i
3.0000 + 1.0000i  4.0000 + 1.0000i
```

```
>> transpose([1+i, 2+i; 3+i, 4+i])
```

```
ans =
```

```
1.0000 + 1.0000i  3.0000 + 1.0000i
2.0000 + 1.0000i  4.0000 + 1.0000i
```

```
>> [1+i, 2+i; 3+i, 4+i]'
```

```
ans =
```

```
1.0000 - 1.0000i  3.0000 - 1.0000i
2.0000 - 1.0000i  4.0000 - 1.0000i
```


Variables

- Direct Assignment and Usage

```
>> V = 8:-1:3;
>> M = [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12];
>> V
V =
     8     7     6     5     4     3
>> M
M =
     1     2     3     4
     5     6     7     8
     9    10    11    12
```

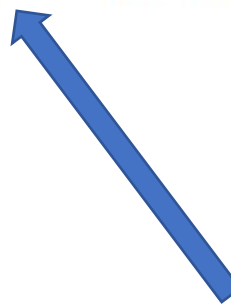
```
>> a = 20260407, b = 333, c = 5.20, d = -1.414, e = 3.1415926, f = pi
```

```
a =
    20260407
b =
    333
c =
    5.2000
d =
   -1.4140
e =
    3.1416
f =
    3.1416
```

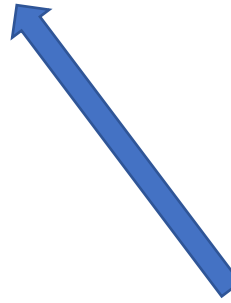
```
>> a2 = 20260407; b2 = 333; c2 = 5.20; d2 = -1.414; e2 = 3.1415926; f2 = pi;
```

```
>> a2, b2, c2, d2, e2, f2
```

```
a2 =
    20260407
b2 =
    333
c2 =
    5.2000
d2 =
   -1.4140
e2 =
    3.1416
f2 =
    3.1416
```



Commands separated by ,



Commands separated by ;

Variables

- Arithmetic Operations

- Addition +

- Subtraction -

- Multiplication *

- Division / or \

- Power ^ or power()

- Root sqrt, ^ or power()

```
>> a = 3; b = 4;
>> a+b
ans =
    7
>> a-b
ans =
   -1
>> a*b
ans =
   12
>> a/b
ans =
  0.7500
>> b\ a
ans =
  0.7500
>> a^b
ans =
   81
```



Retrieval of Variable Properties

- Variable Length

- **length()**

```
>> v1 = [3, 1, 4, 1, 5, 9, 2, 6]; fprintf(' v1='); disp(v1);  
fprintf(' v1 has %d elements\n', length(v1));  
v1=      3      1      4      1      5      9      2      6
```

```
v1 has 8 elements
```

- Matrix Dimensions

```
>> v2 = [2; 7; 1; 8; 3]; fprintf(' v2='); disp(v2);  
fprintf(' v2 has %d elements\n', length(v2));  
v2=      2
```

```
      7
```

```
      1
```

```
      8
```

```
      3
```

```
v2 has 5 elements
```

Retrieval of Variable Properties

- Variable Length
 - **length()**
- Matrix Dimensions
 - **size()**
 - **size(, {DIM})**
 - size(, 1) : Row number
 - size(, 2) : Column number
 - size(, 3) : Channel number
 -

```
>> M = [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]; fprintf('M='); disp(M);  
fprintf('M is a %d-by-%d matrix\n', size(M));  
fprintf('M has %d rows\n', size(M, 1));  
fprintf('M has %d columns\n', size(M, 2));
```

```
M=      1      2      3      4  
      5      6      7      8  
      9     10     11     12
```

```
M is a 3-by-4 matrix  
M has 3 rows  
M has 4 columns
```

Types of Execution (Control Flow)

- Sequential Execution
- Conditional Execution / Selection
- Loop Execution / Looping / Iteration
- Function Call

Types of Execution (Control Flow)

- Sequential Execution

- Separated into different lines

- Command_1
 - Command_2
 - Command_3
 -

- Command_1 ;
 - Command_2 ;
 - Command_3 ;
 -

- Separated by the comma ,

- Command_1, Command_2, Command_3,
 - Displaying execution results

- Separated by the semicolon ;

- Command_1; Command_2; Command_3;
 - Not displaying execution results

```
>> a = 3;
b = 4;
c = power(a^b, 1/2);
fprintf('Integer c = %d\n', c);
fprintf('Floating-point c = %.2f\n', c);
Integer c = 9
Floating-point c = 9.00
```

Types of Execution (Control Flow)

- Conditional Execution

- **if end**

```
>> R = 100;
num = random('uniform', 0, R);
if (num > R/2)
    fprintf('num is in the higher half\n');
end
>> R = 100;
num = random('uniform', 0, R);
if (num > R/2)
    fprintf('num is in the higher half\n');
end
num is in the higher half
```

- **if else end**

```
>> R = 100;
num = random('uniform', 0, R);
if (num > R/2)
    fprintf('num is in the higher half\n');
else
    fprintf('num is in the lower half\n');
end
num is in the higher half
>> R = 100;
num = random('uniform', 0, R);
if (num > R/2)
    fprintf('num is in the higher half\n');
else
    fprintf('num is in the lower half\n');
end
num is in the lower half
```

Types of Execution (Control Flow)

- Conditional Execution

- **if end**

- **if else end**

- **if elseif else end**

```
>> score = random('uniform', 0, 100);  
if (score > 90)  
    fprintf('Score grade is A\n');  
elseif (score > 80)  
    fprintf('Score grade is B\n');  
elseif (score > 60)  
    fprintf('Score grade is PASS\n');  
else  
    fprintf('Score grade is FAIL\n');  
end
```

Score grade is A

```
>> score = random('uniform', 0, 100);  
if (score > 90)  
    fprintf('Score grade is A\n');  
elseif (score > 80)  
    fprintf('Score grade is B\n');  
elseif (score > 60)  
    fprintf('Score grade is PASS\n');  
else  
    fprintf('Score grade is FAIL\n');  
end
```

Score grade is PASS

Types of Execution (Control Flow)

- Conditional Execution
 - **if end**
 - **if else end**
 - **if elseif else end**
 - **switch case otherwise end**
 - Suitable to concrete values instead of ranges

```
>> SG = input('Enter a letter (A/B/P/F):');
switch SG % score grade
    case 'A'
        disp('GOOD');
    case 'B'
        disp('WELL');
    case 'P'
        disp('OK');
    case 'F'
        disp('BAD');
    otherwise
        disp('INVALID INPUT!');
end
Enter a letter (A/B/P/F):'A'
GOOD
```

Types of Execution (Control Flow)

- Loop Execution
 - **for** var={RANGE} **end**
 - **while** {CONDITION} **end**

```
>> sum = 0;
for k=1:100
    sum = sum+k;
end
fprintf('sum = %d\n', sum);
sum = 5050
```

```
>> flg = 1; % Set a judgement flag
while (flg)
    num = random('uniform', 0, 100);
    fprintf('Random num = %.2f\n', num);
    if (num<10) flg = 0; end
end
disp('Loop execution is STOPPED');
Random num = 23.99
Random num = 12.33
Random num = 18.39
Random num = 24.00
Random num = 41.73
Random num = 4.97
Loop execution is STOPPED
```

Types of Execution (Control Flow)

- Loop Execution

- **for** var={RANGE} **end**
- **while** {CONDITION} **end**
- **break** stops the immediate loop

```
>> while (1)
    num = random('uniform', 0, 100);
    fprintf('Random num = %.2f\n', num);
    if (num<10) break; end
end
disp('Loop execution is STOPPED');
```

```
>> sum = 0;
for k=1:100
    sum = sum+k;
end
fprintf('sum = %d\n', sum);
sum = 5050

>> flg = 1; % Set a judgement flag
while (flg)
    num = random('uniform', 0, 100);
    fprintf('Random num = %.2f\n', num);
    if (num<10) flg = 0; end
end
disp('Loop execution is STOPPED');
Random num = 23.99
Random num = 12.33
Random num = 18.39
Random num = 24.00
Random num = 41.73
Random num = 4.97
Loop execution is STOPPED
```

Types of Execution (Control Flow)

- Loop Execution
 - **for** var={RANGE} **end**
 - **while** {CONDITION} **end**
 - **break** stops the immediate loop
- Advice on Matlab Loop Execution
 - **Matlab looping is slow**
 - **Use matrix-oriented processing**
 - Dot . means element-wise processing

```
>> n = 10000; M1 = rand(n,n);  
M2 = rand(n,n); M3 = zeros(size(M1));  
disp('Style 1 starts ...');  
t = cputime;  
for r=1:n  
    for c=1:n  
        M3(r,c) = M1(r,c)*M2(r,c);  
    end  
end  
fprintf('Time used is %.5f\n', cputime-t);  
disp('Style 2 starts ...'); t = cputime;  
M4 = M1.*M2;  
fprintf('Time used is %.5f\n', cputime-t);  
Style 1 starts ...  
Time used is 8.51563  
Style 2 starts ...  
Time used is 0.71875
```

Types of Execution (C

- Function Call
 - **function** out={F}(in) **end**
- At the script end
 - For internal call/invoke
- In a separate script
 - For external call/invoke

```
matlab_demo_func.m x Polynomial_F.m +
1 - fprintf('Polynomial_F1(1)='); disp(Polynomial_F1(1));
2 - fprintf('Polynomial_F1(-1)='); disp(Polynomial_F1(-1));
3 - v = [1;2;3]; % Vector
4 - fprintf('Polynomial_F2(v)='); disp(Polynomial_F2(v));
5 - fprintf('Polynomial_F3(v1, v2, v3)='); disp(Polynomial_F3(v(1), v(2), v(3)));
6 - [y1, y2] = Polynomial_F4(v(1), v(2), v(3));
7 - fprintf('Polynomial_F4 output y1='); disp(y1);
8 - fprintf('Polynomial_F4 output y2='); disp(y2);
9
10 - [y1, y2] = Polynomial_F(v(1), v(2), v(3));
11 - fprintf('Polynomial_F output y1='); disp(y1);
12 - fprintf('Polynomial_F output y2='); disp(y2);
13
14 - function y = Polynomial_F1(x)
15 -     y = x^2 + x + 1;
16 - end
17 - function y = Polynomial_F2(x)
18 -     y = sum(x);
19 -     y = y^2;
20 - end
21 - function y = Polynomial_F3(x1, x2, x3)
22 -     y = x1 + x2 + x3;
23 -     y = y^2;
24 - end
25 - function [y1, y2] = Polynomial_F4(x1, x2, x3)
26 -     y1 = x1 + x2 + x3;
27 -     y1 = y1^2;
28 -     y2 = [-y1, -y1^2, -y1^3];
29 - end
```

```
matlab_demo_func.m x Polynomial_F.m +
1 - function [y1, y2] = Polynomial_F(x1, x2, x3)
2 -     y1 = x1 + x2 + x3;
3 -     y1 = y1^2;
4 -     y2 = [-y1, -y1^2, -y1^3];
5 - end
```

DISCUSSION

Q & A



Matrix Computation

- Matrix Definition Review

- $[A_{11}, \dots, A_{1n}; \dots; A_{m1}, \dots, A_{mn}]$
 - Elements separated by ,
 - Rows separated by ;

- Coding line continuing mark ...

- Transpose

- Attention: For complex matrices, `transpose()` and `'` mean different things

```
>> [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
```

```
ans =
```

```
1     2     3     4
5     6     7     8
9     10    11    12
```

```
>> [1, 2, 3, 4; ...
```

```
5, 6, 7, 8; ...
```

```
9, 10, 11, 12]
```

```
ans =
```

```
1     2     3     4
5     6     7     8
9     10    11    12
```

← ... means continuing the line

```
>> transpose([1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12])
```

```
ans =
```

```
1     5     9
2     6    10
3     7    11
4     8    12
```

```
>> [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]'
```

```
ans =
```

```
1     5     9
2     6    10
3     7    11
4     8    12
```

```
>> [1+i, 2+i; 3+i, 4+i]
```

```
ans =
```

```
1.0000 + 1.0000i  2.0000 + 1.0000i
3.0000 + 1.0000i  4.0000 + 1.0000i
```

```
>> transpose([1+i, 2+i; 3+i, 4+i])
```

```
ans =
```

```
1.0000 + 1.0000i  3.0000 + 1.0000i
2.0000 + 1.0000i  4.0000 + 1.0000i
```

```
>> [1+i, 2+i; 3+i, 4+i]'
```

```
ans =
```

```
1.0000 - 1.0000i  3.0000 - 1.0000i
2.0000 - 1.0000i  4.0000 - 1.0000i
```

Matrix Computation

- Arithmetic Operations
 - Addition +
 - Subtraction -
 - Multiplication *
 - Attention: Dimension consistency

```
>> M1 = [1, 2; 3, 4]; disp(M1)
M2 = [5, 6; 7, 8]; disp(M2)
fprintf('M1+M2 = \n'); disp(M1+M2)
fprintf('M1-M2 = \n'); disp(M1-M2)
fprintf('M1*M2 = \n'); disp(M1*M2)
```

```
1    2
3    4
```

```
5    6
7    8
```

```
M1+M2 =
```

```
6    8
10   12
```

```
M1-M2 =
```

```
-4   -4
-4   -4
```

```
M1*M2 =
```

```
19   22
43   50
```


Matrix Computation

- Arithmetic Operations

- Addition +

- Subtraction -

- Multiplication *

- Square Matrix Operations

- Power ^

- Attention: Not power()

- Inverse $\text{inv}()$ or $\wedge(-1)$

```
>> M = [1, 2; 3, 4]; disp(M)
fprintf(' Square of M = \n'); disp(M^2)
fprintf(' Element-wise square of M = \n'); disp(power(M, 2))

1     2
3     4
```

```
Square of M =
7     10
15     22
```

```
Element-wise square of M =
1     4
9    16
```

```
>> iM = inv(M);
fprintf(' Inverse of M = \n'); disp(iM)
fprintf(' M x iM = \n'); disp(iM*M), disp(M*iM)
Inverse of M =
-2.0000    1.0000
 1.5000   -0.5000
```

Matrix Computation

- Square Matrix Operations

- Power $\wedge$
- Inverse `inv()` or $\wedge(-1)$
- Eigenvalue `eig()`
- Characteristic `poly()`

- Amazing characteristic polynomial

```
>> M = [1, 2; 3, 4];  
eM = eig(M); p = poly(eM);  
p(1)*M^2+p(2)*M+p(3)*eye(2)
```

```
>> [vM, eM] = eig(M); disp([vM, eM])  
-0.8246    -0.4160    -0.3723         0  
0.5658    -0.9094         0    5.3723
```

```
>> eM = eig(M); disp(eM)  
-0.3723  
5.3723
```

```
>> for k=1:length(eM)  
    disp([M*vM(:,k), eM(k)*vM(:,k)])  
end  
0.3070    0.3070  
-0.2106   -0.2106  
  
-2.2347   -2.2347  
-4.8854   -4.8854
```

```
p = poly(eM); disp(p) % p(1)*s^n + ... + p(end-1)*s + p(end)  
1.0000   -5.0000   -2.0000
```



Eigenwert

Matrix Computation

- Linear Equation Solving
 - $xyz = \text{inv}(A) * B$

```
%    x +    y +    z = 3
%    x + 2 y + 3 z = 6
%    x + 4 y + 9 z = 14
```

```
A = [1, 1, 1; ...
      1, 2, 3; ...
      1, 4, 9];
B = [3; 6; 14];
xyz = inv(A)*B;
fprintf('A = \n'); disp(A);
fprintf('B = \n'); disp(B);
fprintf('xyz = \n'); disp(xyz);
```

```
A =
     1     1     1
     1     2     3
     1     4     9
```

```
B =
     3
     6
    14
```

```
xyz =
     1
     1
     1
```

Matrix Computation

- Linear Equation Solving
 - $xyz = A \backslash B$

```
%    x +    y +    z = 3
%    x + 2 y + 3 z = 6
%    x + 4 y + 9 z = 14
```

```
A = [1, 1, 1; ...
      1, 2, 3; ...
      1, 4, 9];
B = [3; 6; 14];
xyz = A \ B;
fprintf('A = \n'); disp(A);
fprintf('B = \n'); disp(B);
fprintf('xyz = \n'); disp(xyz);
```

```
A =
     1     1     1
     1     2     3
     1     4     9
```

```
B =
     3
     6
    14
```

```
xyz =
     1
     1
     1
```

Matrix Computation

- Linear Equation Solving
 - $pv = A \backslash B$

```
% p + 30 v = 30.3
% p + 35 v = 34.9
% p + 40 v = 40.4
% p + 45 v = 45.1
% p + 50 v = 49.5
% p + 55 v = 55.9
% p + 60 v = 59.8
```

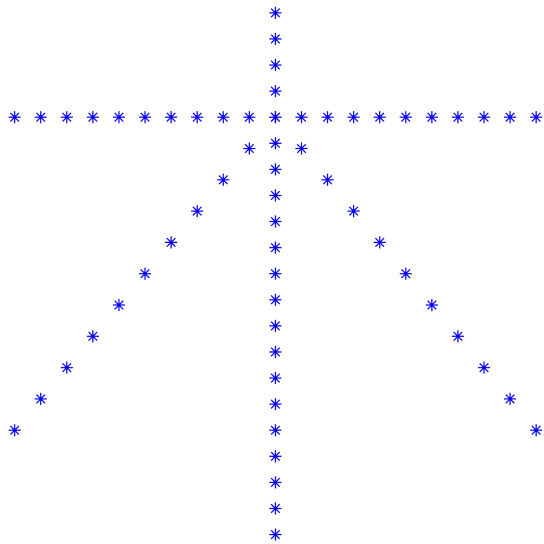
```
>> A = [ones(7,1), (30:5:60)'];
B = [30.3; 34.9; 40.4; 45.1; 49.5; 55.9; 59.8];
fprintf('A = \n'); disp(A);
fprintf('B = \n'); disp(B);
[rank(A), rank([A, B])]
```

A =		B =	ans =	2	3
1	30	30.3000			
1	35	34.9000			
1	40	40.4000			
1	45	45.1000			
1	50	49.5000			
1	55	55.9000			
1	60	59.8000			

```
>> pv = A \ B;
fprintf('pv = \n'); disp(pv);
pv =
    0.2571
    0.9971
```

Matrix Computation

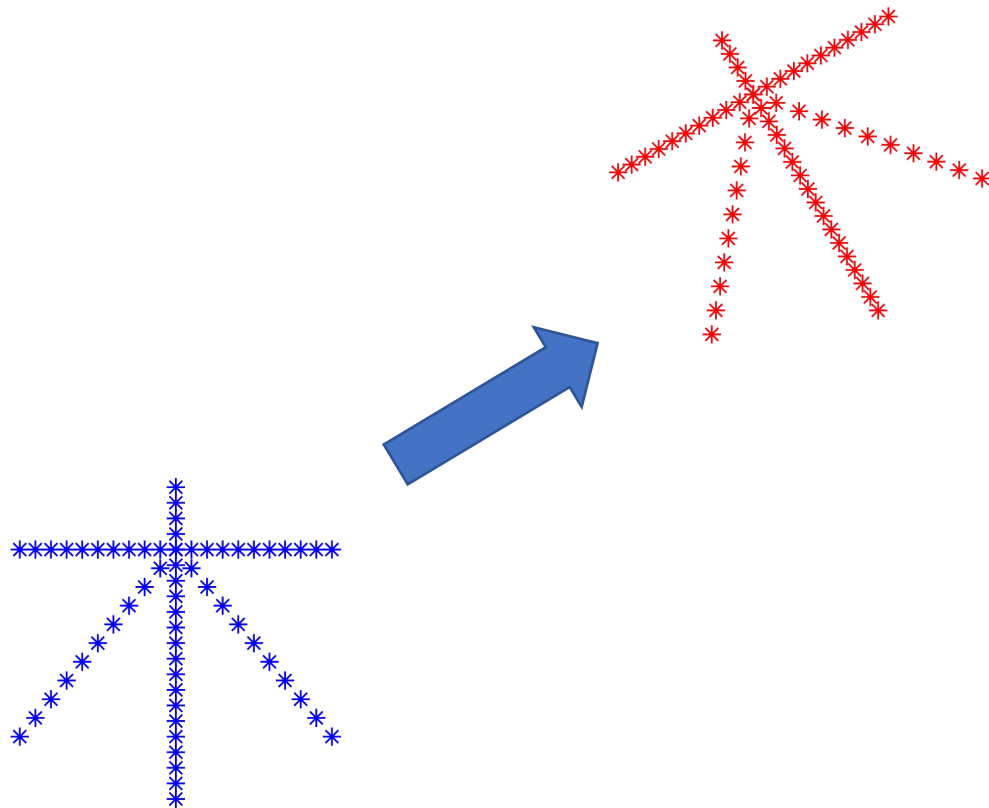
- Matrix-Oriented Processing



```
>> n = 10; r = 0.6;  
xP = [-n:n, zeros(1,2*n+1), -n:0, 0:n];  
yP = [r*n*ones(1,2*n+1), -n:n, r*n+2*r*(-n:0), r*n-2*r*(0:n)];  
figure(1), plot(xP, yP, '*b'); axis equal; axis off;
```

Matrix Computation

- Matrix-Oriented Processing



```
>> a = pi/6; rotM = [cos(a), -sin(a); sin(a), cos(a)];  
xyD = [n*4; n*3];  
fprintf('Rotation matrix = \n'); disp(rotM)  
fprintf('Translation vector = \n'); disp(xyD)
```

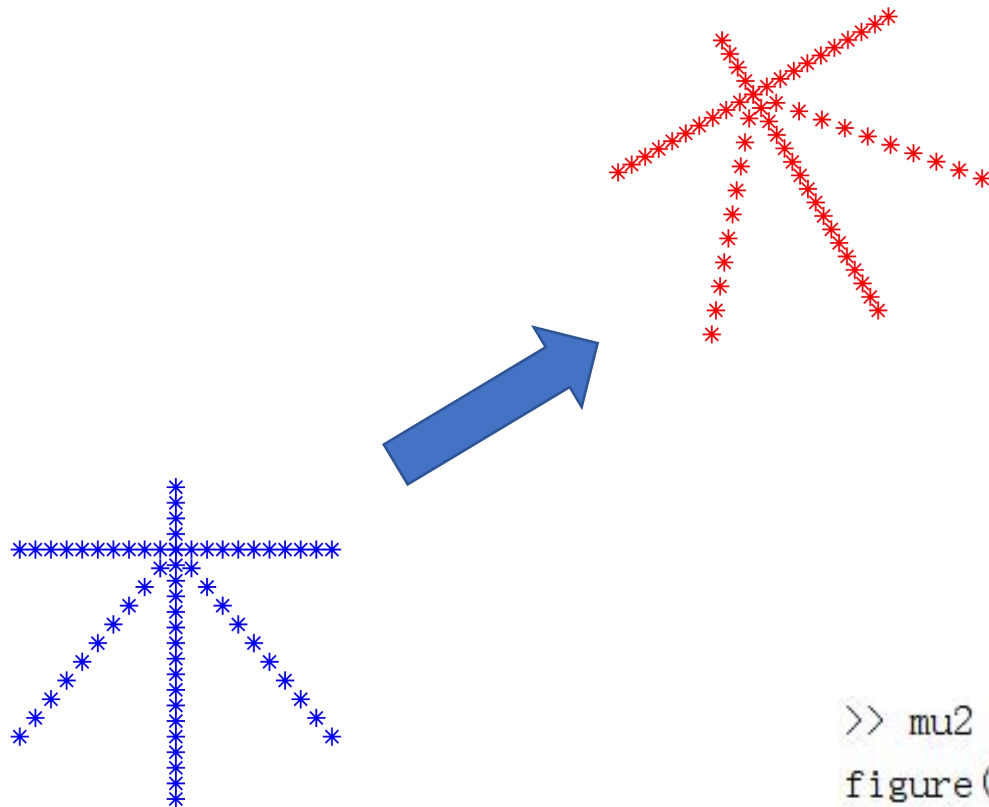
```
Rotation matrix =  
    0.8660   -0.5000  
    0.5000    0.8660
```

```
Translation vector =  
    40  
    30
```

```
for k=1:length(xP)  
    rotM*[xP(k); yP(k)]+xyD;  
end
```


Matrix Computation

- Matrix-Oriented Processing



```
>> a = pi/6; rotM = [cos(a), -sin(a); sin(a), cos(a)];  
xyD = [n*4; n*3];  
fprintf('Rotation matrix = \n'); disp(rotM)  
fprintf('Translation vector = \n'); disp(xyD)
```

```
Rotation matrix =  
    0.8660   -0.5000  
    0.5000    0.8660
```

```
Translation vector =  
    40  
    30
```

```
>> mu2 = rotM*[xP; yP] + xyD*ones(1, length(xP));  
figure(2), plot(mu2(1,:), mu2(2,:), '*r'); axis equal; axis off;  
figure(3), plot(xP, yP, '*b', mu2(1,:), mu2(2,:), '*r');  
axis equal; axis off;
```


DISCUSSION

Q & A

